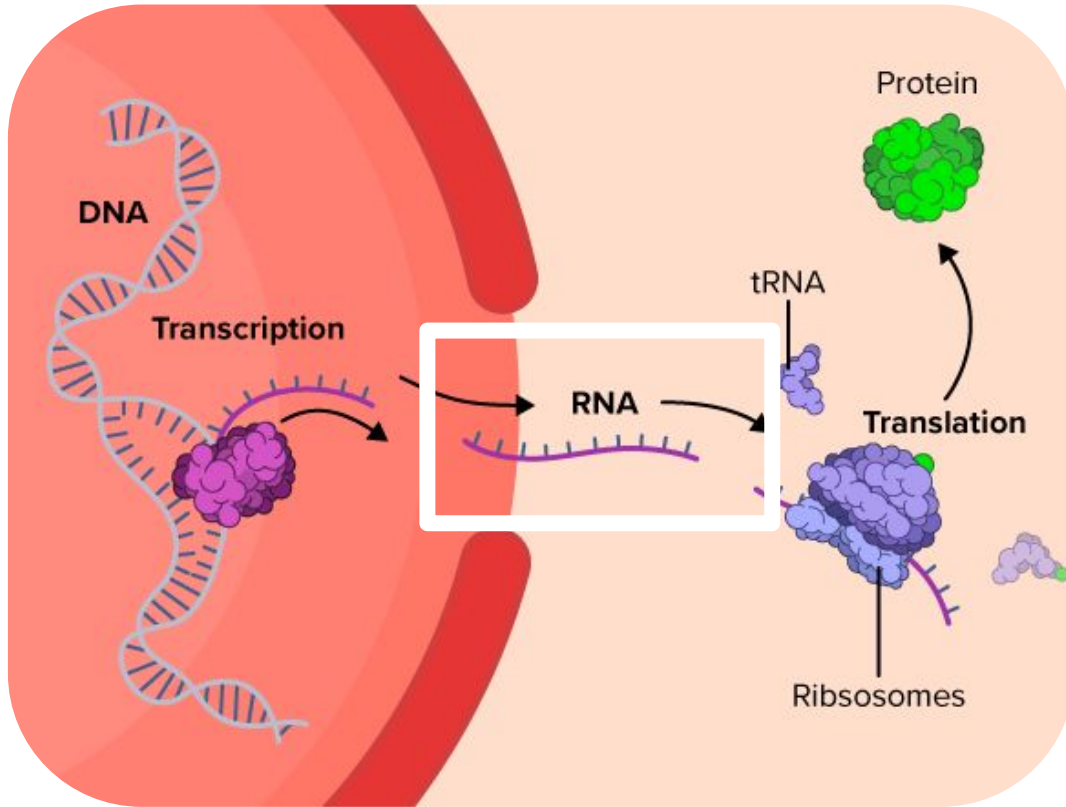


Análisis de RNA-seq

Noemí Gómez

01 de octubre, 2026

noemi.gomez@postgrado.uoh.cl

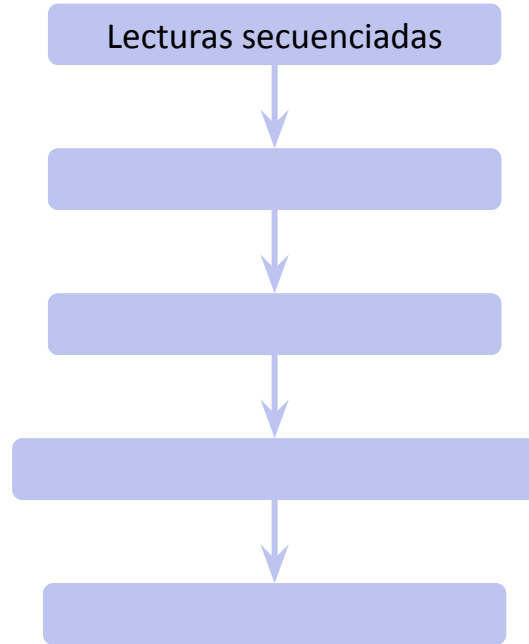


¿Por qué secuenciar ARN?

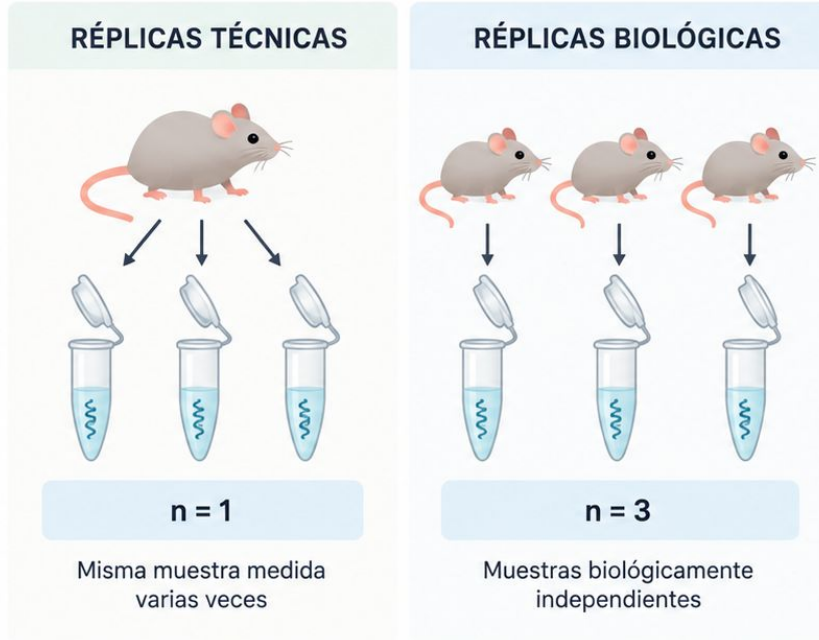
- Determinar qué genes están siendo expresados y en qué cantidad
- Caracterizar el estado funcional
- Comparar distintas condiciones (qué procesos o genes cambian entre ellas)

A partir de estos perfiles podemos estudiar cómo cambia la actividad celular entre condiciones y, posteriormente, relacionar esos cambios con procesos y funciones biológicas.

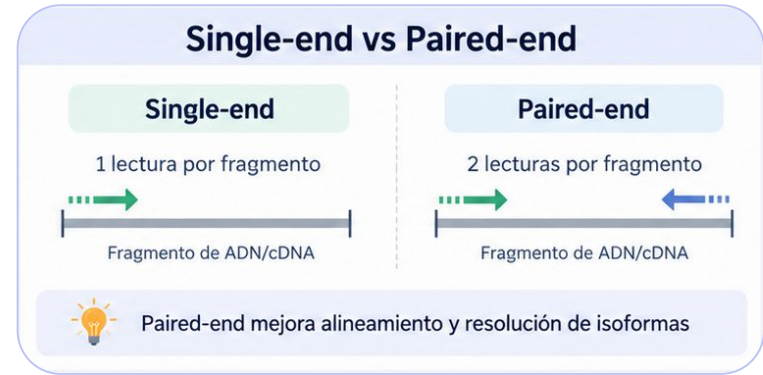
Flujo de trabajo para analizar RNA-seq



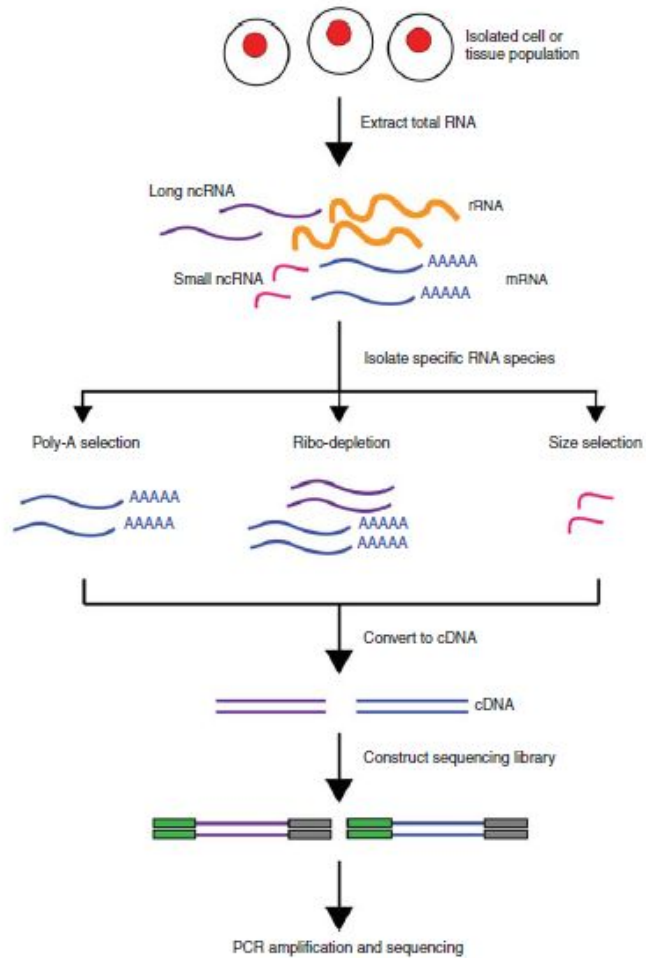
Diseño experimental



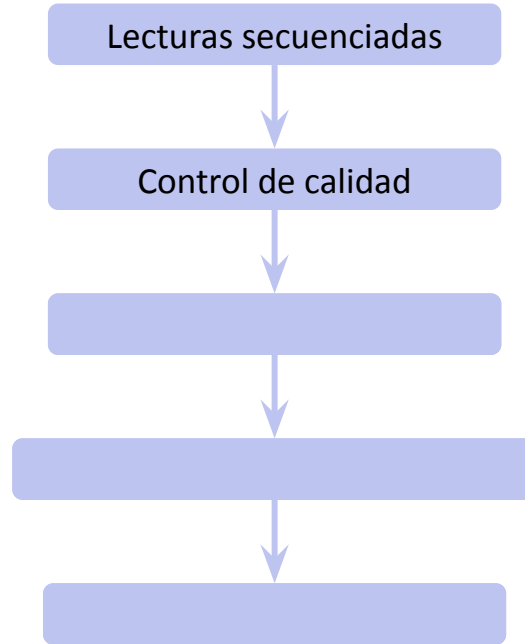
- **Réplicas técnicas:** miden variabilidad del procedimiento.
- **Réplicas biológicas:** capturan variabilidad biológica.
- **Batch effect:** variación introducida por factores técnicos que puede confundirse con variación biológica.



RNA-seq



Flujo de trabajo para analizar RNA-seq



Control de calidad

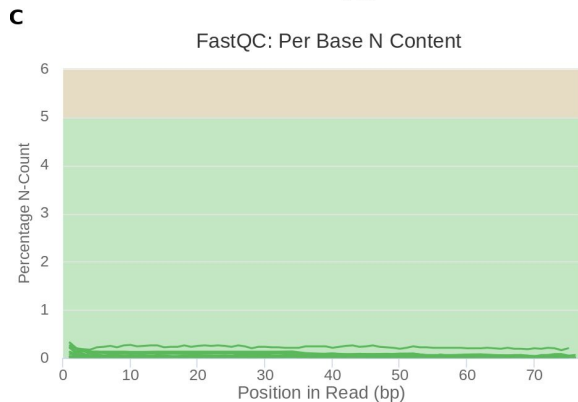
A. Calidad por base

Muestra los scores Phred a lo largo de la lectura. Se espera calidad alta y relativamente estable; una caída al extremo 3' puede requerir trimming.

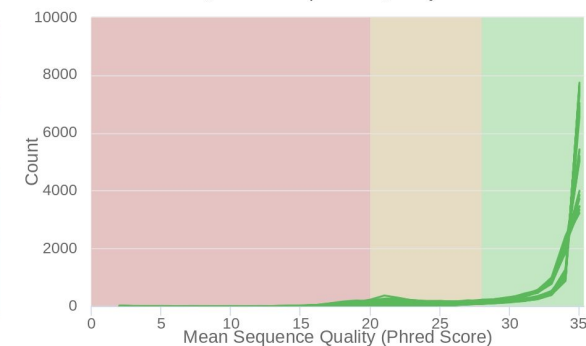


C. Contenido de bases N

Indica bases no identificadas. Debe ser cercano a 0 en toda la lectura.



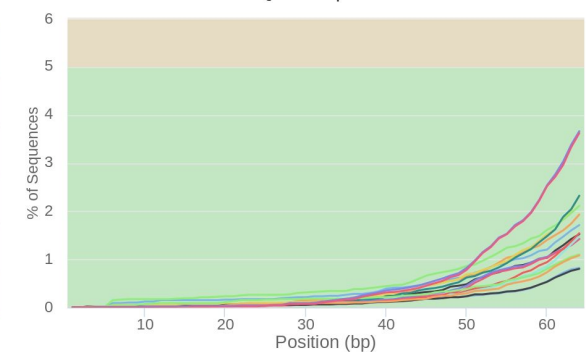
B. Calidad media por secuencia



B. Calidad media por secuencia

Resume la calidad global de las lecturas. Idealmente, la distribución se concentra en scores altos.

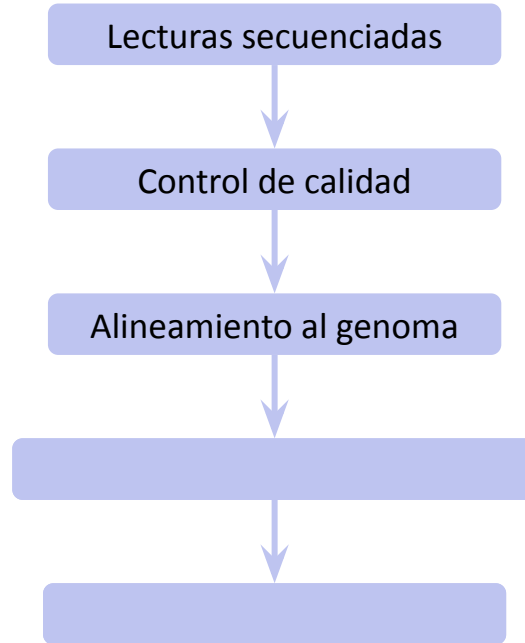
D. Contenido de adaptadores



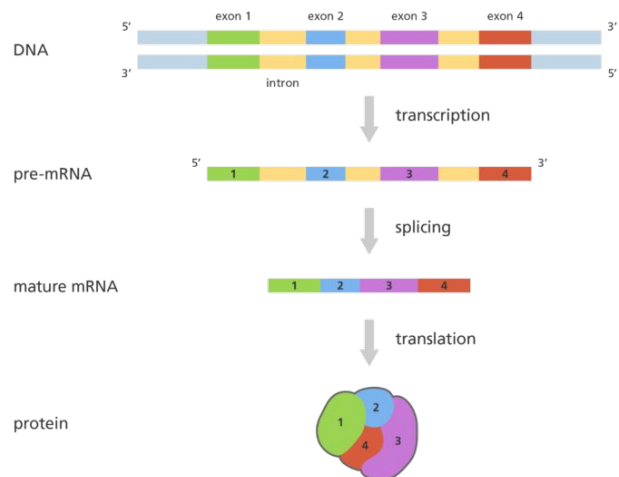
D. Contenido de adaptadores

Detecta secuencias técnicas residuales. Su aumento, especialmente al final, sugiere trimming.

Flujo de trabajo para analizar RNA-seq

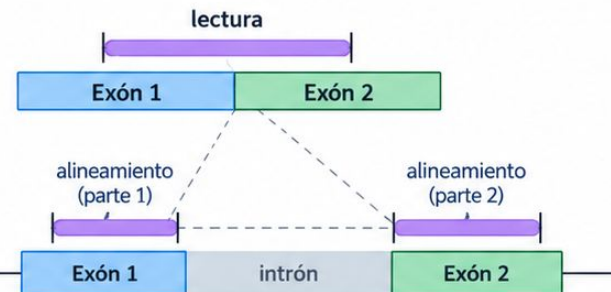


El alineamiento al genoma de referencia es “*splice-aware*”



mRNA maduro
(transcrito)

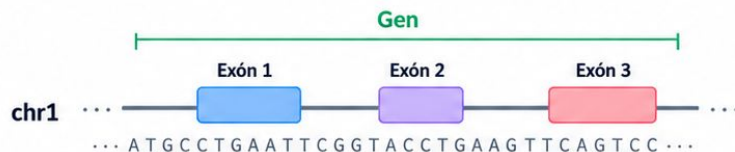
Genoma de referencia
(región del gen)



Una lectura de RNA-seq puede abarcar una **unión exón-exón**.
Por eso se requieren alineadores con **reconocimiento de splicing**.
Ej.: STAR, HISAT2.

¿Contra qué alineamos?

Genoma de referencia

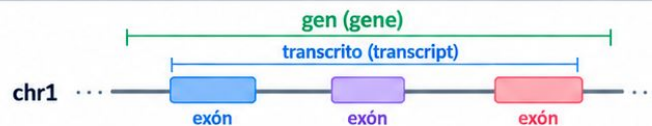


- Secuencia de ADN usada como marco de comparación
- Indica dónde buscar el origen de las lecturas

 Formato típico: **FASTA**

GTF / anotación génica

chr	feature	start	end	gene_id	transcript_id
chr1	gene	1000	9000	gene1	.
chr1	transcript	1000	9000	gene1	tx1
chr1	exon	1000	2000	gene1	tx1
chr1	exon	3000	4000	gene1	tx1
chr1	exon	7000	9000	gene1	tx1
...	...	...	...	...	...



- Archivo con coordenadas de genes, exones y transcritos
- Indica qué significa cada región del genoma

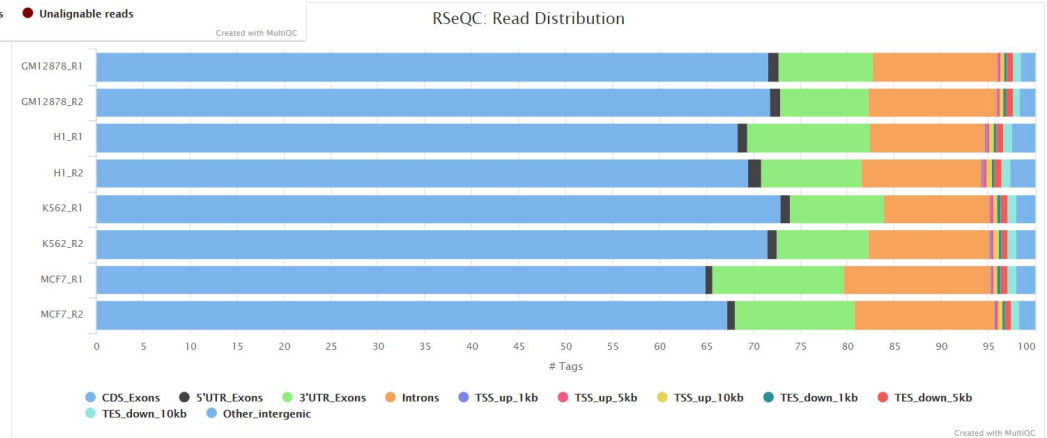
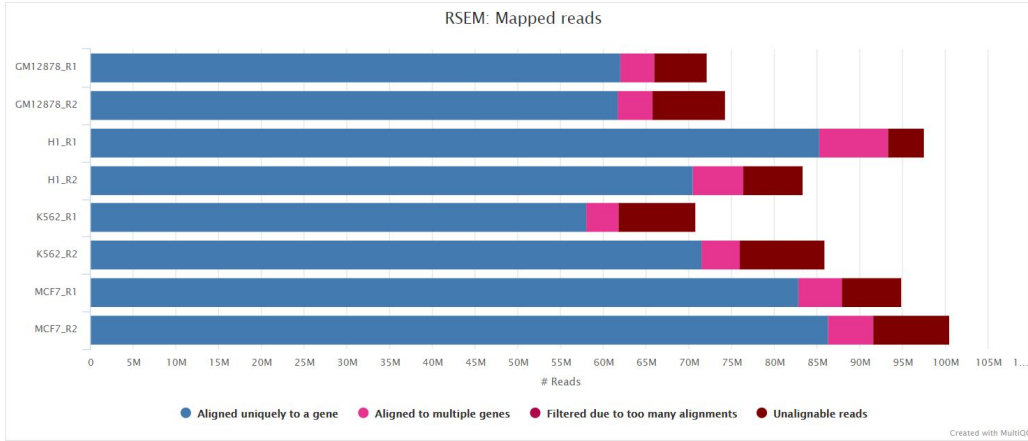
 Ayuda al **alineamiento** y a la **cuantificación**.



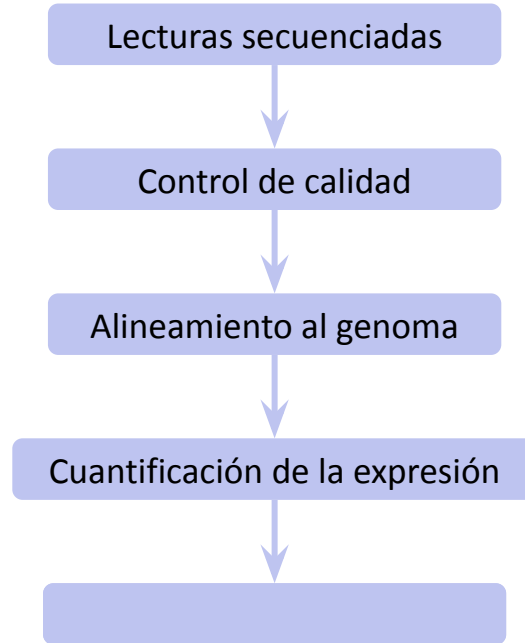
Genoma de referencia humano

Archivo de anotación humano

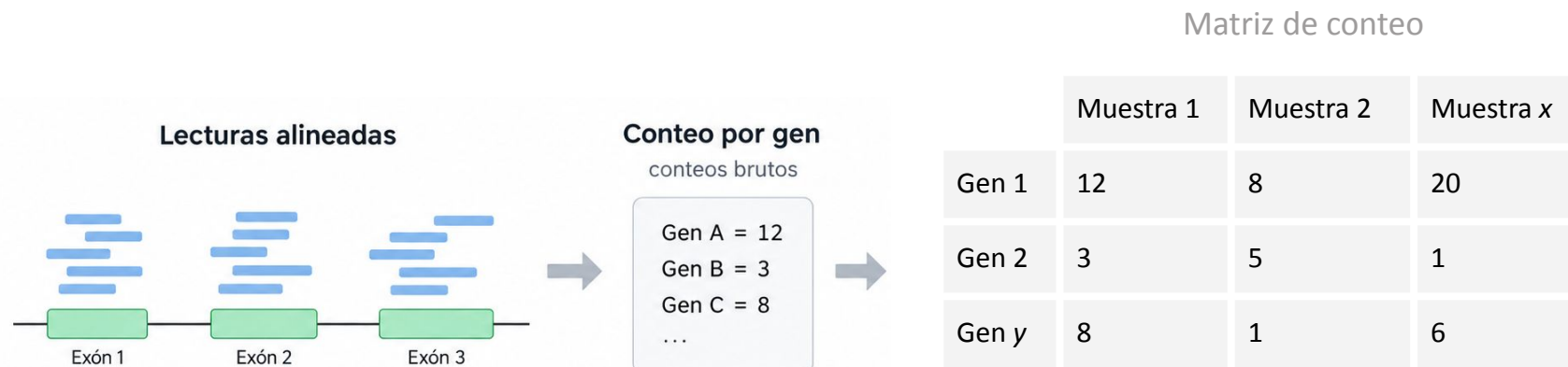
Calidad del alineamiento



Flujo de trabajo para analizar RNA-seq



Cuantificación de la expresión



1

Cada lectura asignada aporta al conteo del gen.

2

El resultado inicial son conteos enteros (raw counts).

3

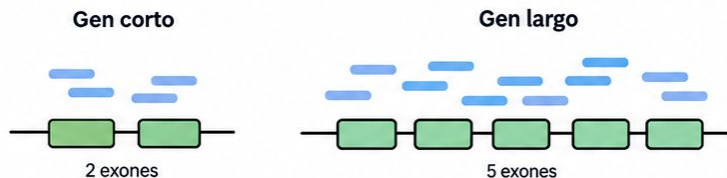
La matriz resume genes × muestras.

La cuantificación transforma lecturas alineadas en una **matriz de conteo** por gen.

Un gen tiene más lecturas que otro, ¿está más expresado?

$$\text{Reads observados} \propto \text{expresión} \times \text{longitud} \times \text{profundidad}$$

1 Efecto de la longitud



A igual expresión, genes más largos pueden acumular más lecturas.

Más longitud → más oportunidad de generar fragmentos

Fragments per kilobase of transcript per million fragments mapped



FPKM

- corrige por longitud + profundidad
- abundancia normalizada
- menos usado actualmente

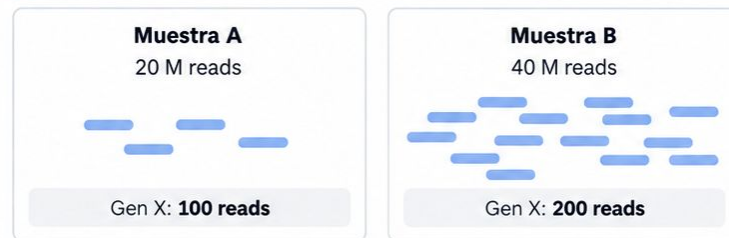
Transcripts per million



TPM

- corrige por longitud + profundidad
- abundancia relativa
- útil para comparar genes dentro de muestra

2 Efecto de la profundidad

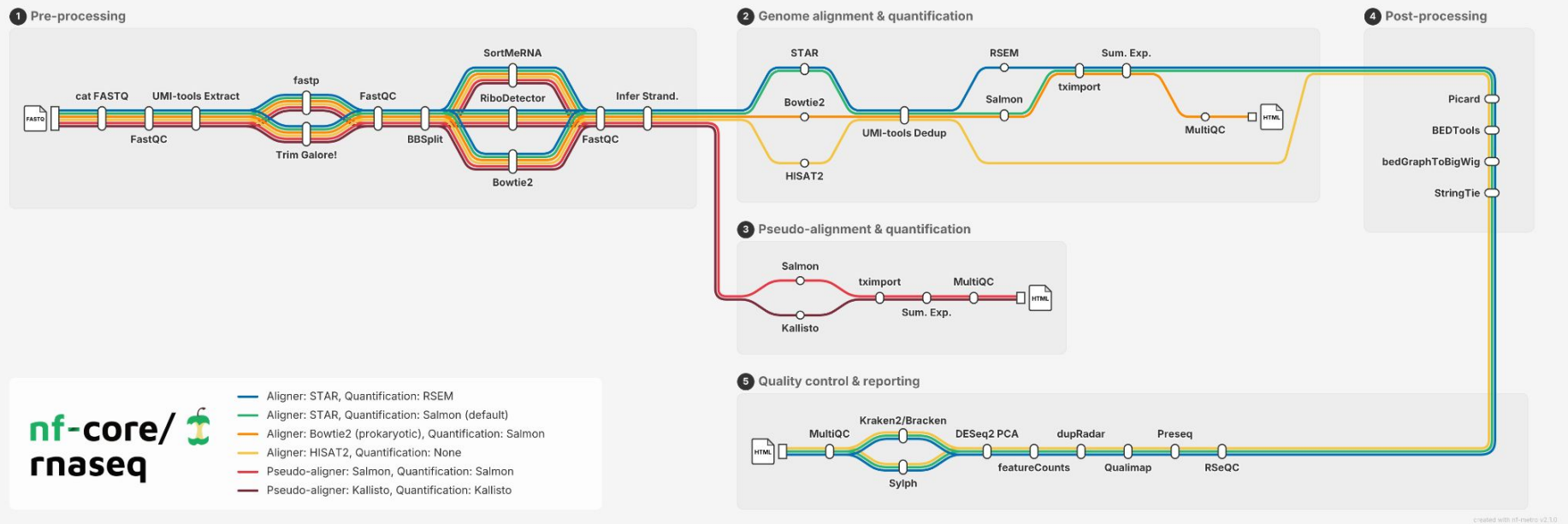


A igual expresión, una biblioteca más profunda produce más lecturas observadas.

Más profundidad → más reads por muestra

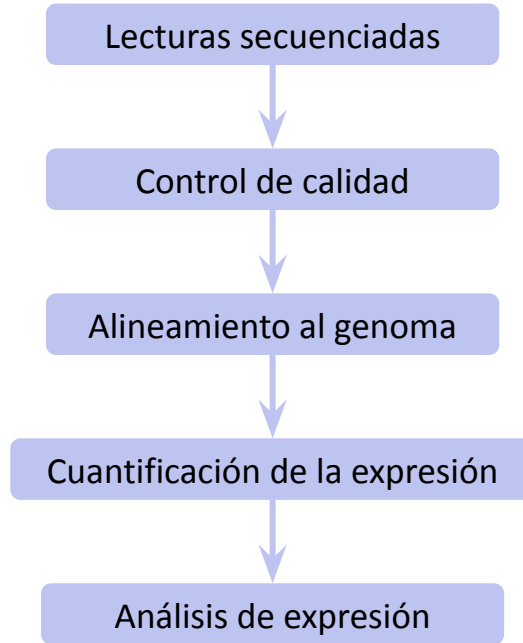
Más lecturas no significan siempre más expresión, puede influir la longitud del gen y profundidad de secuenciación.

Un flujo de trabajo para lecturas cortas



<https://github.com/nf-core/rnaseq>

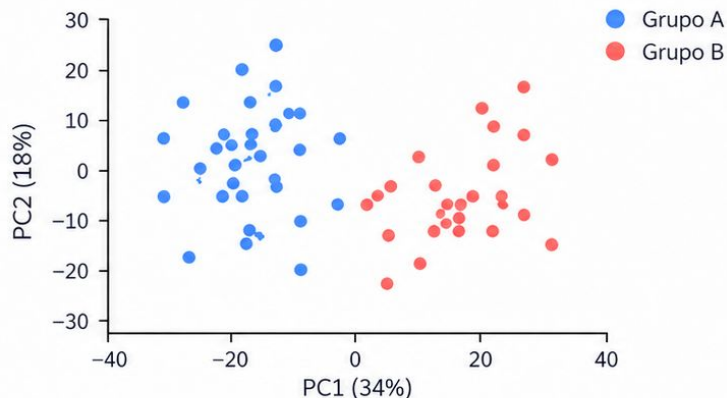
Flujo de trabajo para analizar RNA-seq



Análisis no supervisado

Explora la estructura global de los datos sin definir grupos previos

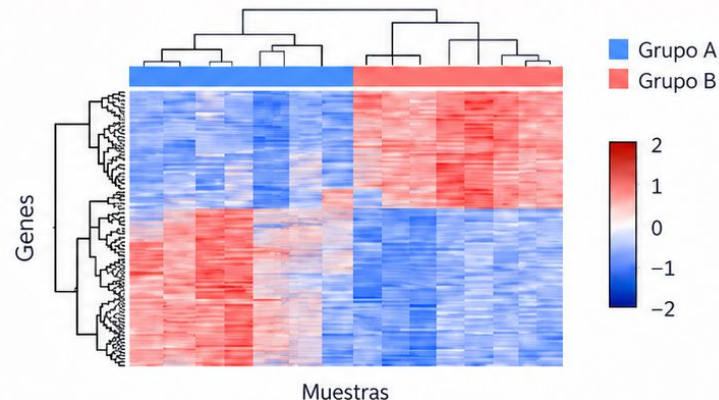
PCA



- Resume la variación global entre muestras.
- Si las muestras se agrupan, pueden identificarse grupos o subgrupos.
- Permite detectar outliers y posibles fuentes de variación.

Uso: visión global rápida

Clustering



- Agrupa muestras o genes con perfiles similares.
- Permite identificar subgrupos y patrones de expresión.
- Complementa la visión global entregada por PCA.

Uso: relaciones detalladas entre muestras y genes

Normalización

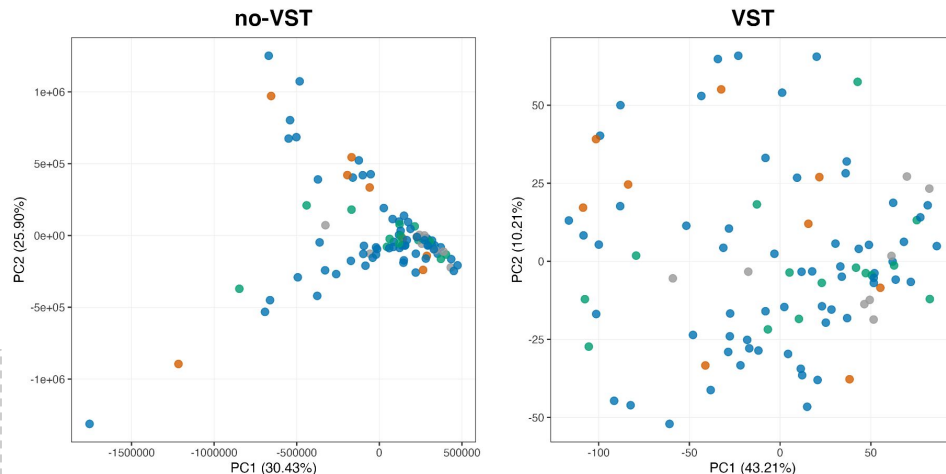
Hace comparables las muestras y prepara los datos para análisis exploratorios.

- Corrige diferencias globales entre bibliotecas.
- **VST**: variance stabilizing transformation. Una función en DESeq2 en R que reduce la dependencia entre media y varianza.
- Útil para análisis no supervisados.

```
library(DESeq2)
```

```
dds <- DESeqDataSetFromMatrix(  
  countData = round(rna),  
  colData = metadata,  
  design = ~ 1)  
vsd <- vst(dds, blind = TRUE)  
rna_vst <- assay(vsd)
```

“Intenta evitar que un gen muy expresado tenga automáticamente una influencia mucho mayor simplemente porque sus conteos son más grandes.”



Genes muy abundantes pueden dominar las diferencias. Con VST, la contribución de genes queda equilibrada, facilitando encontrar patrones globales biológicos.

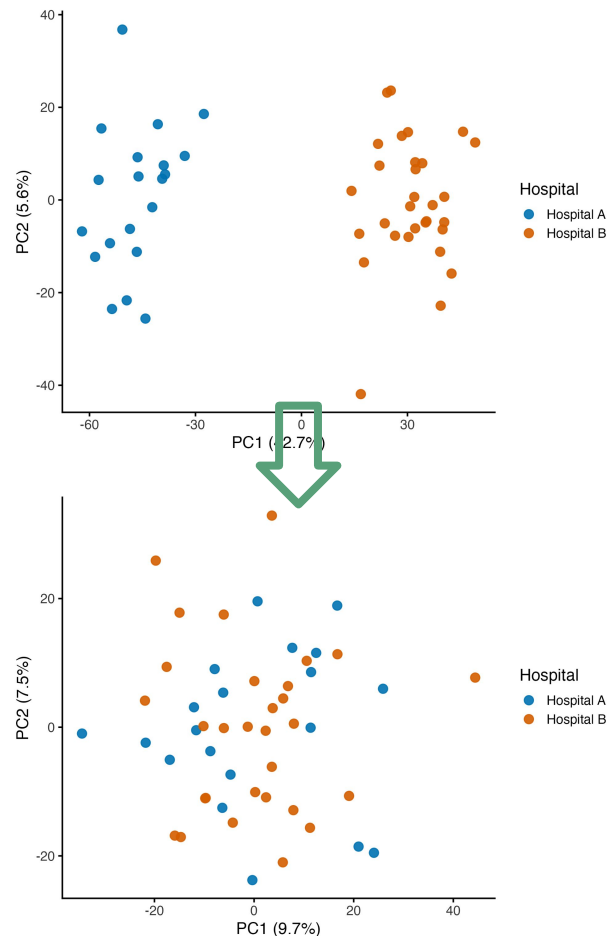
Control de *batch effect*

Cuando la variación entre muestras se explica por una variante técnica

- Puede generar agrupamientos no biológicos
- Se corrige para evitar confundir señal técnica con biológica

Métodos

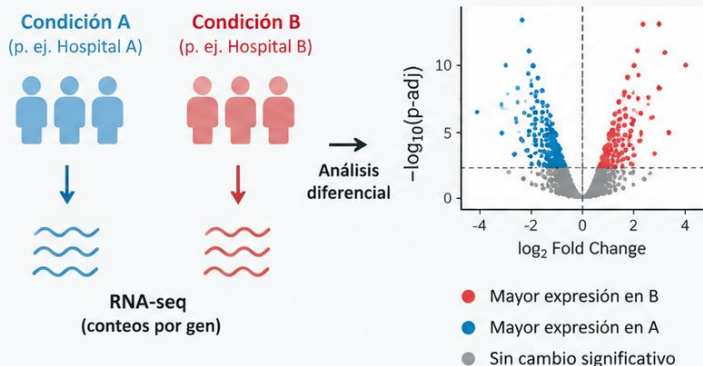
- `limma::removeBatchEffect()`
- `sva::ComBat_seq()`
- DESeq2-MultiBatch cuando hay más de un sesgo técnico
- Incluir como covariable para capturar su varianza en modelos multivariables
 - $\sim \text{batch} + \text{condición}$



Análisis de expresión diferencial

¿Qué es?

Identificar genes cuya expresión cambia entre condiciones o grupos de muestras, considerando la variabilidad entre muestras.



Conceptos clave

- Log2FoldChange: magnitud y dirección del cambio de expresión entre condiciones.
 - $\text{Log2FC} > 0$ = mayor expresión en B.
 - $\text{Log2FC} < 0$ = mayor expresión en A.
- p-value: probabilidad de que un evento se deba al azar. *Se suele filtrar por un valor < 0.05*
- FDR/p-adj: p-value ajustado por múltiples pruebas. *Tasa de falsos positivos. Se suele filtrar por < 0.05 .*
- Covariables: factores adicionales que se incluyen para evitar confusiones. Por ejemplo: sexo, pureza tumoral, *batch*.

Paquetes en R: **DESeq2 (estándar)**, edgeR, limma-voom.

¿Cómo hacer un análisis de expresión diferencial con DESeq2?

https://github.com/noemigomezr/rnaseq_class

Actividad práctica: realizar un análisis de expresión diferencial en adenocarcinoma de colon de acuerdo a estado de duplicación de genoma completo

```
install.packages("BiocManager")
```

```
install.packages("tidyverse")
```

```
install.packages("ggplot2")
```

```
BiocManager::install("DESeq2")
```

```
library(tidyverse)
```

```
library(ggplot2)
```

```
library(DESeq2)
```

```
rna <- read.table(file="../input/coad_rna.tsv", sep="\t", header=TRUE, check.names = FALSE, row.names  
= 1)
```

```
metadata <- read.table(file="../input/coad_metadata.tsv", sep="\t", header = TRUE)
```

```
head(metadata)
```

```
dim(rna)
```

```
# El modelo DESeq2 necesita una colData que tenga en las filas las muestras, y la columna con el valor del diseño
```

```
coldata <- metadata %>% select(PATIENT_ID, wholeGenomeDuplication) %>%  
  column_to_rownames(var = "PATIENT_ID") %>%  
  mutate(wholeGenomeDuplication = as.factor(wholeGenomeDuplication))
```

```
table(coldata)
```

```
wholeGenomeDuplication  
false  true  
  178   122
```

```
# eliminar genes sin expresión  
table(rowSums(rna) > 0) # TRUE genes con expresión, FALSE genes donde en todas las muestras son 0  
rna <- rna[rowSums(rna) > 0, ]  
dim(rna)
```

```
FALSE  TRUE  
12252 48408  
[1] 48408   300
```

```
dds <- DESeqDataSetFromMatrix(countData = rna,
                              colData = coldata,
                              design = ~wholeGenomeDuplication)
```

```
dds <- DESeq(dds)
resultsNames(dds)
```

```
[1] "Intercept"
[2] "wholeGenomeDuplication_true_vs_false"
```

```
res <- results(dds, contrast = c("wholeGenomeDuplication", "true", "false")) %>% as.data.frame()
summary(res)
```

baseMean		log2FoldChange		lfcSE		stat		pvalue		padj	
Min.	: 0.00	Min.	:-4.813	Min.	:0.000	Min.	:-6.183	Min.	:0.000	Min.	:0.00
1st Qu.:	0.00	1st Qu.:	-0.122	1st Qu.:	0.194	1st Qu.:	-0.340	1st Qu.:	0.345	1st Qu.:	0.50
Median :	0.05	Median :	-0.005	Median :	0.490	Median :	-0.002	Median :	0.772	Median :	0.75
Mean :	16.47	Mean :	-0.002	Mean :	1.134	Mean :	-0.040	Mean :	0.647	Mean :	0.67
3rd Qu.:	1.80	3rd Qu.:	0.129	3rd Qu.:	2.406	3rd Qu.:	0.239	3rd Qu.:	0.968	3rd Qu.:	0.90
Max. :	33246.05	Max. :	3.846	Max. :	2.952	Max. :	5.719	Max. :	1.000	Max. :	1.00
		NA's	:21144	NA's	:21144	NA's	:21144	NA's	:21144	NA's	:44237

```
# log2FC > 0 - enriquecido en true, < 0 enriquecido en false
# Un log2FC de 1 indica un cambio de dos veces. Es decir, si es 1 entonces en esa condicion esta
expresado dos veces mas.
true_enriched <- res %>% filter(log2FoldChange > 1, padj < 0.1)
false_enriched <- res %>% filter(log2FoldChange < -1, padj < 0.1)

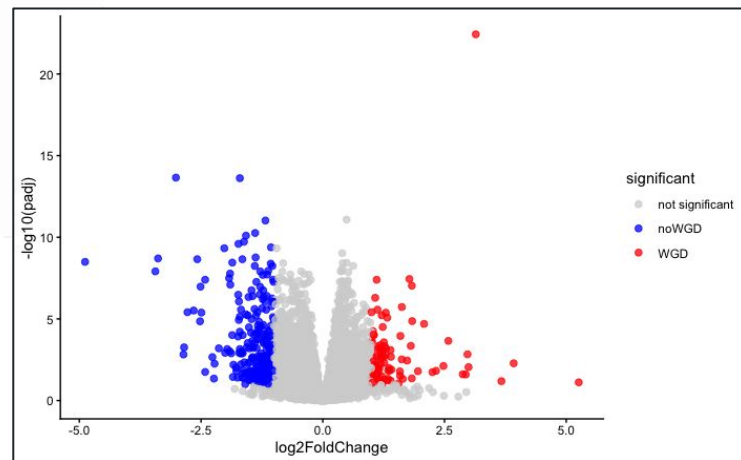
cat("Hay", nrow(true_enriched), "genes sobreexpresados en duplicacion de genoma completo, versus",
nrow(false_enriched), "subexpresados.")
```

Hay 101 genes sobreexpresados en
duplicacion de genoma completo, versus
246 subexpresados.

Graficamos los genes diferencialmente expresados en un *volcano plot*

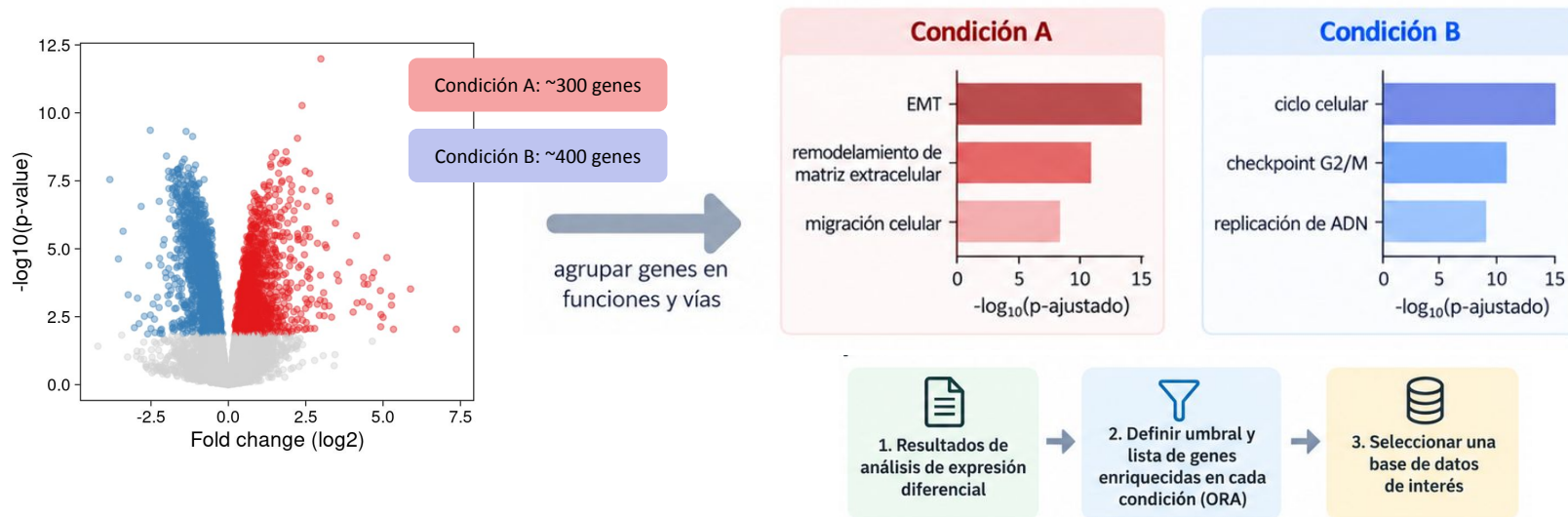
```
# Definir colores
res <- res %>%
  mutate(gene = rownames(res),
         significant = ifelse(gene %in% rownames(true_enriched), "WGD",
                             ifelse(gene %in% rownames(false_enriched),
                                    "noWGD",
                                    "not significant"))))

# generar volcano
ggplot(res, aes(x = log2FoldChange, y = -log10(padj),
               color = significant)) +
  geom_point(alpha = 0.75, size = 2) +
  scale_color_manual(values = c("WGD" = "red", "noWGD" = "blue",
                              "not significant" = "lightgray")) +
  theme_classic()
```



Enriquecimiento funcional

Un análisis de expresión diferencial puede identificar cientos de genes por condición. En lugar de revisar por separado, el enriquecimiento funcional resume estos cambios en procesos biológicos y vías alteradas de acuerdo al conjunto de genes diferencialmente expresados.



El objetivo no es solo saber qué genes cambian, sino qué funciones biológicas representan esos cambios.

MSigDB

Base de datos que organiza genes en conjuntos definidos por una función, vía o característica biológica común.



Permite interpretar una lista o ranking de genes en términos de funciones y procesos biológicos.



Hallmark: programas biológicos representativos



GO: procesos biológicos, función molecular y componente celular



Pathways: Reactome, KEGG y otras vías



Firmas: inmunológicas, oncogénicas, regulatorias y de tipos celulares

```
BiocManager::install("msigdb")  
library(msigdb)
```

<https://www.gsea-msigdb.org/gsea/msigdb/collections.jsp>

The screenshot shows the MSigDB website interface. At the top, there's a navigation bar with links like 'GSEA Home', 'Downloads', 'Molecular Signatures Database', 'Documentation', 'Contact', and 'Team'. Below this, the main heading is 'Human MSigDB Collections'. To the left, there's a sidebar with links for 'MSigDB Home', 'Human Collections' (with sub-links: About, Browse, Search, Investigate, Gene Families), 'Mouse Collections' (with sub-links: About, Browse, Search, Investigate), and 'Help'. The main content area explains that 35361 gene sets are divided into 9 major collections. It includes a table with three rows: 'H: hallmark gene sets' (50 gene sets), 'C1: positional gene sets' (302 gene sets), and 'C2: curated gene sets' (7670 gene sets). Each row provides a brief description and links to download GMT files, Gene Symbols, NCBI (Entrez) Gene IDs, and a JSON bundle.

Collection	Description	Download Options
H: hallmark gene sets (browse 50 gene sets)	Hallmark gene sets summarize and represent specific well-defined biological states or processes and display coherent expression. These gene sets were generated by a computational methodology based on identifying overlaps between gene sets in other MSigDB collections and retaining genes that display coordinate expression. details	Download GMT Files Gene Symbols NCBI (Entrez) Gene IDs JSON bundle
C1: positional gene sets (browse 302 gene sets)	Gene sets corresponding to human chromosome cytogenetic bands. details	Download GMT Files Gene Symbols NCBI (Entrez) Gene IDs JSON bundle
C2: curated gene sets (browse 7670 gene sets)	Gene sets in this collection are curated from various sources, including online pathway databases and the biomedical literature. Many sets are also contributed by individual domain experts. The gene set page for each gene set lists its source. The C2 collection is divided into the following two subcollections: Chemical and genetic perturbations (CGP) and Canonical pathways (CP). details	Download GMT Files Gene Symbols NCBI (Entrez) Gene IDs JSON bundle

¿Cómo hacer un análisis de enriquecimiento?

ORA

Over-Representation Analysis

- Usa una lista de genes significativos
- Evalúa si una función o vía está sobrerrepresentada
- Útil cuando ya definimos un umbral

```
BiocManager::install("clusterProfiler")  
  
library(clusterProfiler)  
  
clusterProfiler::enricher()
```

GSEA

Gene Set Enrichment Analysis

- Usa el ranking completo de genes
- No requiere un corte arbitrario de significancia
- Detecta enriquecimiento en los extremos del ranking

```
BiocManager::install("fgsea")  
library(fgsea)  
fgsea::fgsea()  
library(clusterProfiler)  
clusterProfiler::GSEA()
```

Análisis ORA con clusterProfiler

```
BiocManager::install("clusterProfiler")  
BiocManager::install("msigdbR")
```

```
library(clusterProfiler)  
library(msigdbR)
```

```
# Definir una base de datos. Por ejemplo = GO procesos biológicos  
go_bp <- msigdbR(species = "Homo sapiens",  
                 collection = "C5",  
                 subcollection = "G0:BP")
```

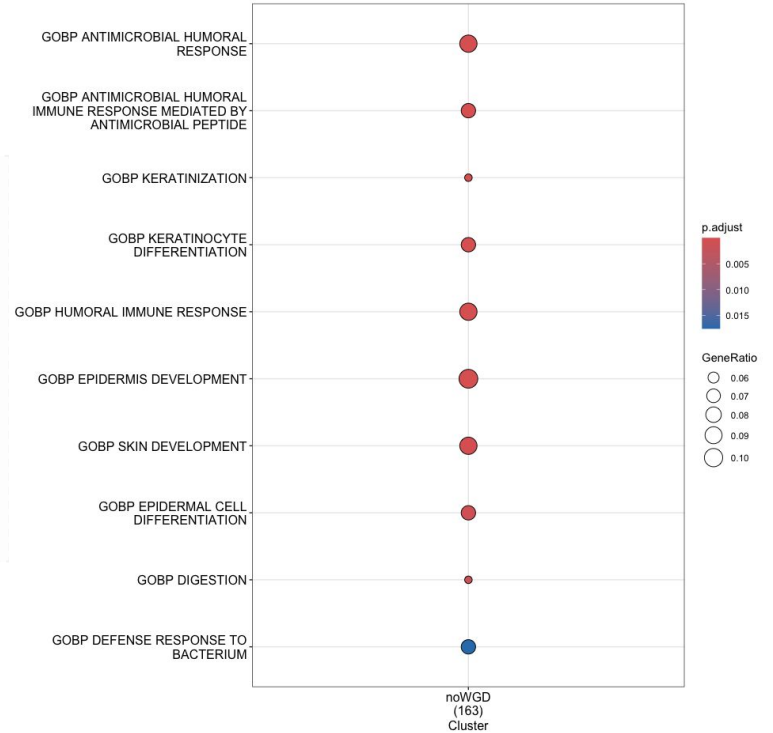
```
# El universo de genes a estudiar debe ser los genes utilizados en la prueba  
universo <- res$gene
```

```
# Se establece una tabla de asociacion proceso-gen  
term2gene <- unique(as.data.frame(go_bp[, c("gs_name", "ensembl_gene"])))
```

```
genes <- list("WGD" = res$gene[res$significant == "WGD"],
             "noWGD" = res$gene[res$significant == "noWGD"])

enrich <- compareCluster(geneClusters = genes,
                        fun = "enricher",
                        TERM2GENE = term2gene,
                        universe = universo,
                        pAdjustMethod = "BH")

dotplot(enrich, showCategory = 10,
        color = "p.adjust")
```



Actividad práctica: la continuación

- DESeq2 permite definir covariables del modelo en su parámetro *design*. ¿Cómo cambia al agregar pureza?
- Realizar un PCA y clustering jerárquico con los datos de BRCA. Luego, caracterizar los cluster (K=2) mediante análisis de expresión diferencial y enriquecimiento funcional. ¿Se relacionan los cluster con la metadata?
- Opcional: repetir análisis en COAD cambiando la base de datos y/o la columna de evaluación
 - Por ejemplo: metástasis utilizando KEGG, CMS utilizando hallmark

Cómo obtener y cargar un gtf

```
# En terminal
wget https://ftp.ebi.ac.uk/pub/databases/gencode/Gencode_human/release_50/gencode.v50.annotation.gtf.gz
```

```
# En R
BiocManager::install("rtracklayer")
library(rtracklayer)

gtf <- import("gencode.v50.annotation.gtf.gz")
library(tidyverse)

gene_chr <- as.data.frame(gtf) %>%
  filter(type == "gene") %>%
  transmute(
    ensembl = sub("\\..*$", "", gene_id),
    chromosome = as.character(seqnames)
  ) %>%
  distinct()
gene_chr <- gene_chr %>%
  filter(chromosome %in% c(paste0("chr", 1:22))) # solo autosomas
```

Cómo obtener y cargar un gtf

```
# Directo en R
BiocManager::install("biomaRt")

library(biomaRt)
library(tidyverse)

mart <- useEnsembl(
  biomaRt = "genes",
  dataset = "hsapiens_gene_ensembl"
)

gene_chr <- getBM(
  attributes = c(
    "ensembl_gene_id",
    "chromosome_name"
  ),
  mart = mart
) %>%
  filter(chromosome_name %in% c(1:22)) %>%
  distinct()
```

```
# Varianza en R
var <- apply(df, 1, var)# varianza por fila
sort(var, decreasing = TRUE)[1:5000] # ordenar de mayor a menor y seleccionar las primeras 5000

# PCA en R
prcomp(t(df)) # considerando que df es gen x muestra (fila x columna)
pca$x # contiene los componentes por muestra

# clustering jerarquico
h <- hclust(dist(pca$x))
cluster <- cutree(hc, k = 2) # corta en dos cluster

# fisher test (categorico x categorico)
fisher_res <- sapply(fisher_vars, \(v)
  fisher.test(table(meta$cluster, metadata_brca[[v]]))$p.value)

# wilcoxon (categorico x continuo)
wilcox_res <- sapply(wilcox_vars, \(v)
  wilcox.test(meta[[v]] ~ metadata_brca$cluster)$p.value)
```

Análisis de RNA-seq

Noemí Gómez

01 de octubre, 2026

noemi.gomez@postgrado.uoh.cl